

AD23211 - PYTHON FOR DATA SCIENCE

UNIT II SETS, DICTIONARIES, FILES, PACKAGES

9

Sets : Operations and methods; Dictionaries: operations and methods; Files and exceptions: text files, reading and writing files, format operator; command line arguments, Errors and exceptions, handling exceptions, Modules, Packages; Illustrative Programs.

PYTHON SETS

1. Describe in detail about dictionary methods. (or) What is Dictionary? Give an example. **(8 MARKS) (DEC 2018)**
2. Give a function that can take a value and return the first key mapping to that value in a Dictionary. **(2 MARKS) (JAN 2019)**
3. Define Dictionary in python. Do the following operations on dictionaries? **(10 MARKS) (NOV 2019)**
 - i) Initialize two dictionaries with key and value pairs.
 - ii) Compare two dictionaries with master key list and print missing keys.
 - iii) Find keys that are in first and not in second dictionary.
 - iv) Find same keys in two dictionaries.
 - v) Merge two dictionaries and create a new dictionary using a single expression.
4. How python dictionaries store data? Give example. **(2 MARKS) (DEC 2018)**
5. Write a python program to create a dictionary and sort the contents based on the values in reverse order. **(8 MARKS) (JAN 2022)**

A set is a mutable data type that allows you to store immutable types in an unsorted way. Sets are mutable because you can add and remove items from them. They can contain immutable items, like tuples and other primitive types, but not lists, sets, or dictionaries which are themselves mutable. Unlike a list or a tuple, a set can only contain one instance of a unique item. There are no duplicates allowed.

Definition

- A set is an unordered collection of unique elements.
- Defined using curly braces {} or the set() function.

Characteristics

- Unordered: No indexing or slicing.
- Mutable: Can add or remove elements.
- Unique: No duplicate elements.

Creating Sets

Using curly braces

```
my_set = {1, 2, 3, 4}
```

Using the set() function

```
another_set = set([1, 2, 3, 4])
```

Adding Elements

```
my_set.add(5) # Adds a single element
```

```
my_set.update([6, 7, 8]) # Adds multiple elements
```

Removing Elements

```
my_set.remove(3) # Raises KeyError if element not found
```

```
my_set.discard(4) # Does not raise an error if element not found
```

```
my_set.pop() # Removes and returns an arbitrary element
```

```
my_set.clear() # Removes all elements
```

Set Operations

The set operations are performed on two or more sets to obtain a combination of elements as per the operation performed on them. In a set theory, there are three major types of operations performed on sets, such as

1. Union of sets ($\cup$)
2. Intersection of sets ($\cap$)
3. Difference of sets ($-$)

Syntax:

Set operations

```
union_set = set1 | set2 # Union of sets
```

```
intersection_set = set1 & set2 # Intersection of sets
```

```
difference_set = set1 - set2 # Set difference
```

```
symmetric_difference_set = set1 ^ set2 # Symmetric difference
```

Union: Combines elements from both sets.

```
set1 = {1, 2, 3}
```

```
set2 = {3, 4, 5}
```

```
union_set = set1 | set2 # {1, 2, 3, 4, 5}
```

Intersection: Elements common to both sets.

```
intersection_set = set1 & set2 # {3}
```

Difference: Elements in one set but not the other.

```
difference_set = set1 - set2 # {1, 2}
```

Symmetric Difference: Elements in either set, but not both.

```
sym_diff_set = set1 ^ set2 # {1, 2, 4, 5}
```

Membership Testing

```
3 in my_set # True
```

```
10 in my_set # False
```

Iterating Through a Set

```
for element in my_set:
```

```
    print(element)
```

Set Comprehensions

```
squared_set = {x**2 for x in range(10)} # {0, 1, 4, 9, 16, 25, 36, 49, 64, 81}
```

Sets are useful for storing unique items and performing mathematical set operations.

They provide efficient membership testing and are highly versatile for various applications.

Sets: Methods

Syntax:

```
# Set methods
```

```
my_set.add(element) # Add an element to the set
```

```
my_set.remove(element) # Remove an element from the set (raises KeyError if  
element not found)
```

```
my_set.discard(element) # Remove an element from the set if present (no error if  
element not found)
```

```
my_set.clear() # Remove all elements from the set
```

```
my_set.update(iterable) # Add multiple elements to the set
```

```
my_set.intersection_update(iterable) # Update the set with the intersection of itself  
and another set
```

```
my_set.difference_update(iterable) # Update the set with the difference of itself and  
another set
```

```
my_set.symmetric_difference_update(iterable) # Update the set with the symmetric  
difference of itself and another set
```

Example Program:

```
# Example program demonstrating set methods
```

```
# Creating a set
```

```
my_set = {1, 2, 3}
```

```
# Set methods
```

```
my_set.add(4)
```

```
my_set.remove(2)
my_set.discard(5) # No error raised if element not found
my_set.clear() # Update with multiple elements
my_set.update({5, 6, 7}) # Print the updated set print("Updated Set:", my_set)
```

Output:

Updated Set: {5, 6, 7} In this example, we create a set my_set and demonstrate various methods such as add(), remove(), discard(), clear(), and update().

Each method is applied to the set, and the updated set is printed to illustrate the effect of each method.

Method	Description	Example
set.add(x)	Add an element to this set	>>> s = {1, 2, 3} >>> s.add(4) # {1, 2, 3, 4}
set.clear()	Remove all elements from this set	>>> s = {1, 2, 3} >>> s.clear() # set()
set.copy()	Create and return a flat copy of this set	>>> s = {1, 2, 'Alice'} >>> s.copy() # Returns: {1, 2, 'Alice'}
set.difference(x)	Return a new set with elements of this set except the ones in the given set arguments.	>>> {1, 2, 3}.difference({1, 2}) {3}
set.difference_update(iter)	Remove all elements from this set that are members of any of the given set arguments.	>>> s = {1, 2, 3} >>> s.difference_update({1, 2}) # s == {3}
set.discard(x)	Remove an element from this set if it is a member, otherwise do nothing.	>>> s = {'Alice', 'Bob', 'Cloe'} >>> s.discard('Bob') # s == {'Alice', 'Cloe'}
set.intersection()	Return a new set of elements that are members of this and the set argument(s).	>>> {1, 2, 3, 4}.intersection({3, 4, 5}) {3, 4}

set.intersection_update()	Removes all elements from this set that are not members in all other specified sets.	<pre>>>> s = {1, 2, 3, 4} >>> s.intersection_update({3, 4, 5}) # s == {3, 4}</pre>
set.isdisjoint(x)	Return True if their <u>intersection is the empty set</u>.	<pre>>>> {1, 2, 3, 4}.isdisjoint({'Alice', 'Bob'}) True</pre>
set.issubset()	Return True if all elements of this set are members of the specified set argument.	<pre>>>> t = {'Alice', 'Bob', 'Carl', 'Liz'} >>> {'Alice', 'Bob'}.issubset(t) True</pre>
set.issuperset()	Return True if all elements of the specified set argument are members of this set.	<pre>>>> {'Alice', 'Bob', 'Carl'}.issuperset({'Alice'}) True</pre>
set.pop()	Remove and return a random element from this set. KeyError if set is empty.	<pre>>>> s = {'Alice', 'Bob', 'Carl'} >>> s.pop() 'Alice'</pre>
set.remove()	Remove and return a specific element from this set as defined in the argument. If the set doesn't contain element, raise KeyError.	<pre>>>> s = {'Alice', 'Bob', 'Cloe'} >>> s.remove('Bob') # s == {'Alice', 'Cloe'}</pre>
set.symmetric_difference()	Return new set with elements in either this or the specified set argument, but not both.	<pre>>>> {1, 2, 3}.symmetric_difference({2, 3, 4}) {1, 4}</pre>

set.symmetric_difference_update()	Replace this set with the symmetric difference, i.e., elements in either this set or the specified set argument, but not both.	<pre>>>> s = {1, 2, 3} >>> s.symmetric_difference_update({2, 3, 4}) >>> s {1, 4}</pre>
set.union()	Create and return new set with all elements in this or any of the specified sets.	<pre>>>> {1, 2, 3, 4}.union({3, 4, 5}) {1, 2, 3, 4, 5}</pre>
set.update()	Update this set with all elements that are in this or any of the specified set arguments.	<pre>>>> s = {1, 2, 3, 4} >>> s.update({3, 4, 5}) # s == {1, 2, 3, 4, 5}</pre>

DICTIONARIES:

dictionary is a versatile and powerful data structure that allows you to store key-value pairs.

- Dictionary is an unordered collection of elements. An element in dictionary has a key: value pair.
- All elements in dictionary are placed inside the curly braces i.e. { }
- Elements in Dictionaries are **accessed via keys** and not by their position.
- The values of a dictionary can be any data type.
- Keys must be immutable data type (numbers, strings, tuple)

Creating a Dictionary

- You can create a dictionary using curly braces { } or the dict() function.

Using curly braces

```
my_dict = {
    'name': 'Alice',
    'age': 25,
    'city': 'Hyderabad'
}
```

Using the dict() function

```
my_dict = dict(name='Alice', age=25, city='Hyderabad')
```

Accessing Values

- You can access the values in a dictionary by using the keys.

```
print(my_dict['name']) # Output: Alice
```

```
print(my_dict.get('age')) # Output: 25
```

Adding or Updating Entries

- You can add new key-value pairs or update existing ones.

```
my_dict['email'] = 'alice@example.com' # Adding a new key-value pair
```

```
my_dict['age'] = 26 # Updating an existing key-value pair
```

Removing Entries

- You can remove entries using the del statement or the pop() method.

```
del my_dict['city'] # Removes the key 'city'
```

```
age = my_dict.pop('age') # Removes the key 'age' and returns its value
```

Iterating Through a Dictionary

- You can iterate through keys, values, or key-value pairs.

Iterating through keys

```
for key in my_dict:
```

```
    print(key)
```

Iterating through values

```
for value in my_dict.values():
```

```
    print(value)
```

Iterating through key-value pairs

for key, value in my_dict.items():

```
print(f'{key}: {value}')
```

Dictionary Comprehensions

- You can create dictionaries using comprehensions for more concise and readable code.

```
squares = {x: x*x for x in range(6)}
```

```
print(squares) # Output: {0: 0, 1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
```

Common Methods

- Here are some common dictionary methods:

`my_dict.keys()` - Returns a view object of all keys.

`my_dict.values()` - Returns a view object of all values.

`my_dict.items()` - Returns a view object of all key-value pairs.

`my_dict.clear()` - Removes all items from the dictionary.

`my_dict.update(other_dict)` - Updates the dictionary with key-value pairs from another dictionary.

Operations on dictionary:

1. Accessing an element
2. Update
3. Add element
4. Membership

Operations	Example	Description
Creating a dictionary	<pre>>>> a={1:"one",2:"two"} >>> print(a) {1: 'one', 2: 'two'}</pre>	Creating the dictionary with elements of different data types.
accessing an element	<pre>>>> a[1] 'one' >>> a[0] KeyError: 0</pre>	Accessing the elements by using keys.
Update	<pre>>>> a[1]="ONE" >>> print(a) {1: 'ONE', 2: 'two'}</pre>	Assigning a new value to key. It replaces the old value by new value.
add element	<pre>>>> a[3]="three" >>> print(a) {1: 'ONE', 2: 'two', 3: 'three'}</pre>	Add new element in to the dictionary with key.
membership	<pre>a={1: 'ONE', 2: 'two', 3: 'three'} >>> 1 in a True >>> 3 not in a False</pre>	Returns True if the key is present in dictionary. Otherwise returns false.

Methods in dictionary:

Method	Example	Description
a.copy()	<pre>a={1: 'ONE', 2: 'two', 3: 'three'} >>> b=a.copy() >>> print(b) {1: 'ONE', 2: 'two', 3: 'three'}</pre>	It returns copy of the dictionary. here copy of dictionary 'a' get stored in to dictionary 'b'
a.items()	<pre>>>> a.items() dict_items([(1, 'ONE'), (2, 'two'), (3, 'three')])</pre>	Return a new view of the dictionary's items. It displays a list of dictionary's (key, value) tuple pairs.
a.keys()	<pre>>>> a.keys() dict_keys([1, 2, 3])</pre>	It displays list of keys in a dictionary
a.values()	<pre>>>> a.values() dict_values(['ONE', 'two', 'three'])</pre>	It displays list of values in dictionary
a.pop(key)	<pre>>>> a.pop(3) 'three' >>> print(a) {1: 'ONE', 2: 'two'}</pre>	Remove the element with <i>key</i> and return its value from the dictionary.
setdefault(key,value)	<pre>>>> a.setdefault(3,"three") 'three' >>> print(a) {1: 'ONE', 2: 'two', 3: 'three'} >>> a.setdefault(2) 'two'</pre>	If key is in the dictionary, return its value. If key is not present, insert key with a value of dictionary and return dictionary.
a.update(dictionary)	<pre>>>> b={4:"four"} >>> a.update(b) >>> print(a) {1: 'ONE', 2: 'two', 3: 'three', 4: 'four'}</pre>	It will add the dictionary with the existing dictionary
fromkeys()	<pre>>>> key={"apple","ball"} >>> value="for kids" >>> d=dict.fromkeys(key,value) >>> print(d) {'apple': 'for kids', 'ball': 'for kids'}</pre>	It creates a dictionary from key and values.
len(a)	<pre>a={1: 'ONE', 2: 'two', 3: 'three'} >>> len(a) 3</pre>	It returns the length of the list.
clear()	<pre>a={1: 'ONE', 2: 'two', 3: 'three'} >>>a.clear() >>>print(a) >>>{ }</pre>	Remove all elements form the dictionary.
del(a)	<pre>a={1: 'ONE', 2: 'two', 3: 'three'} >>> del(a)</pre>	It will delete the entire dictionary.

Difference between List, Tuples and dictionary:

List	Tuples	Dictionary
A list is mutable	A tuple is immutable	A dictionary is mutable
Lists are dynamic	Tuples are fixed size in nature	In values can be of any data type and can repeat, keys must be of immutable type
List are enclosed in brackets [] and their elements and size can be changed	Tuples are enclosed in parenthesis () and cannot be updated	Tuples are enclosed in curly braces { } and consist of key:value
Example: List = [10, 12, 15]	Example: Words = ("spam", "eggs") Or Words = "spam", "eggs"	Example: Dict = {"ram": 26, "abi": 24}
Access: print(list[0])	Access: print(words[0])	Access: print(dict["ram"])
Can contain duplicate elements	Can contain duplicate elements. Faster compared to lists	Cant contain duplicate keys, but can contain duplicate values
Slicing can be done	Slicing can be done	Slicing can't be done
Usage: ❖ List is used if a collection of data that doesn't need random access. ❖ List is used when data can be modified frequently	Usage: ❖ Tuple can be used when data cannot be changed. ❖ A tuple is used in combination with a dictionary i.e. a tuple might represent a key.	Usage: ❖ Dictionary is used when a logical association between key:value pair. ❖ When in need of fast lookup for data, based on a custom key. ❖ Dictionary is used when data is being constantly modified.

FILES AND EXCEPTIONS

1. Write a method to rename and delete files. **(2 MARKS) (NOV 2018)**
2. How to use command line arguments in python? **(2 MARKS) (NOV 2019)** Write a function that copies a file reading and writing up to 50 characters at a time. (or) Explain the commands used to read and write into a file with example. **(16 MARKS) (JAN 2019)**
3. Tabulate the different modes for opening a file and briefly explain the same. **(16 MARKS) (JAN 2018) (JAN 2019)**
4. Write the syntax for opening a file to write in binary mode. **(2 MARKS) (JAN 2022)**
5. Write the program to concatenate the contents of two file into a single file. Get the input from the user and concatenate it. **(8 MARKS) (JAN 2022)**

FILE HANDLING

- File is a named location on disk to store related information. It is used to permanently store data in a non-volatile memory (e.g. hard disk).
- Since, random access memory (RAM) is volatile which loses its data when computer is turned off, we use files for future use of the data.
- When we want to read from or write to a file we need to open it first. When we are done, it needs to be closed, so that resources that are tied with the file are freed. Hence, in Python, a file operation takes place in the following order.
 1. Open a file
 2. Read or write (perform operation)
 3. Close the file

Opening a File

- You can open a file using the `open()` function.
- This function requires at least one argument: the file name.
- You can also specify the mode in which the file should be opened.
- This function returns a file object, also called a handle, as it is used to read or modify the file accordingly.

```
>>> f = open("test.txt") # open file in current directory
```

```
>>> f = open("C:/Python33/README.txt") # specifying full path
```

- We can specify the mode while opening a file. In mode, we specify whether we want to read 'r', write 'w' or append 'a' to the file. We also specify if we want to open the file in text mode or binary mode.
- The default is reading in text mode. In this mode, we get strings when reading from the file. On the other hand, binary mode returns bytes and this is the mode to be used when dealing with non-text files like image or exe files.

Method	Description
<code>close()</code>	Closes the file
<code>detach()</code>	Returns the separated raw stream from the buffer
<code>fileno()</code>	Returns a number that represents the stream, from the operating system's perspective
<code>flush()</code>	Flushes the internal buffer
<code>isatty()</code>	Returns whether the file stream is interactive or not
<code>read()</code>	Returns the file content
<code>readable()</code>	Returns whether the file stream can be read or not

readline()	Returns one line from the file
readlines()	Returns a list of lines from the file
seek()	Change the file position
seekable()	Returns whether the file allows us to change the file position
tell()	Returns the current file position
truncate()	Resizes the file to a specified size
writable()	Returns whether the file can be written to or not
write()	Writes the specified string to the file
writelines()	Writes a list of strings to the file

Python File Modes

Mode: Description

- 'r' : Open a file for reading. (default)
- 'w' : Open a file for writing. Creates a new file if it does not exist or truncates the file if it exists.
- 'x' : Open a file for exclusive creation. If the file already exists, the operation fails.
- 'a' : Open for appending at the end of the file without truncating it. Creates a new file if it does not exist.
- 't' : Open in text mode. (default)
- 'b' : Open in binary mode.
- '+' : Open a file for updating (reading and w

f = open("test.txt") # equivalent to 'r' or 'rt'

f = open("test.txt", 'w') # write in text mode

f = open("img.bmp", 'r+b') # read and write in binary mode

- Hence, when working with files in text mode, it is highly recommended to specify the encoding type.

f = open("test.txt", mode = 'r', encoding = 'utf-8')

Open a file in read mode

```
file = open('example.txt', 'r')
```

Open a file in write mode

```
file = open('example.txt', 'w')
```

Open a file in append mode

```
file = open('example.txt', 'a')
```

Open a file in binary mode

```
file = open('example.txt', 'rb')
```

Reading from a File

- There are several methods to read from a file:

Read the entire file

with open('example.txt', 'r') as file:

```
    content = file.read()
```

```
    print(content)
```

Read line by line

with open('example.txt', 'r') as file:

```
    for line in file:
```

```
        print(line.strip())
```

Read a specific number of characters

with open('example.txt', 'r') as file:

```
    content = file.read(10)
```

```
    print(content)
```

Writing to a File

- You can write to a file using the write() method. If the file does not exist, it will be created.

Write to a file

with open('example.txt', 'w') as file:

```
file.write('Hello, World!\n')
```

Append to a file

with open('example.txt', 'a') as file:

```
file.write('Appending a new line.\n')
```

Closing a File

- When we are done with operations to the file, we need to properly close it.
- Closing a file will free up the resources that were tied with the file and is done using the close() method.
- Python has a garbage collector to clean up unreferenced objects but, we must not rely on it to close the file.

```
f = open("test.txt",encoding = 'utf-8')
```

- It's important to close a file after you're done with it to free up system resources.

```
file = open('example.txt', 'r')
```

Perform file operations

```
file.close()
```

Using the with statement is a better practice as it automatically closes the file for you:

with open('example.txt', 'r') as file:

```
content = file.read()
```

```
print(content)
```

Example: Reading and Writing Combined

- Here's a simple example that reads from one file and writes its content to another file:

Read from source file and write to destination file

with open('source.txt', 'r') as source_file:

```
content = source_file.read()
```

with open('destination.txt', 'w') as destination_file:

```
destination_file.write(content)
```

TEXT FILES:

- Python provides inbuilt functions for creating, writing, and reading files. There are two types of files that can be handled in python, normal text files and binary files (written in binary language, 0s, and 1s).
 1. Text files: In this type of file, each line of text is terminated with a special character called EOL (End of Line), which is the new line character ('\n') in python by default.
 2. Binary files: In this type of file, there is no terminator for a line, and the data is stored after converting it into machine-understandable binary language

READING FROM A FILE:

- There are three ways to read data from a text file.
 1. **read()** : Returns the read bytes in form of a string. Reads n bytes, if no n specified, reads the entire file.
`File_object.read([n])`
 2. **readline()** : Reads a line of the file and returns in form of a string. For specified n, reads at most n bytes. However, does not read more than one line, even if n exceeds the length of the line.
`File_object.readline([n])`
 3. **readlines()** : Reads all the lines and return them as each line a string element in a list.
`File_object.readlines()`

Note: '\n' is treated as a special character of two bytes

Example:

Program to show various ways to read and # write data in a file.

```
file1 = open("myfile.txt", "w")
L = ["This is Delhi \n", "This is Paris \n", "This is London \n"]

# \n is placed to indicate EOL (End of Line)
file1.write("Hello \n")
file1.writelines(L)
file1.close() # to change file access modes

file1 = open("myfile.txt", "r+")

print("Output of Read function is ")
print(file1.read())
```



```
file1.seek(0)
print("Output of Readline function is ")
print(file1.readline())
print() file1.seek(0)
# To show difference between read and readline
print("Output of Read(9) function is ")
print(file1.read(9))
print() file1.seek(0)
print("Output of Readline(9) function is ") print(file1.readline(9))

file1.seek(0)
# readlines function
print("Output of Readlines function is ") print(file1.readlines())
print()
file1.close()
```

Output:

Output of Read function is

Hello

This is Delhi

This is Paris

This is London

Output of Readline function is

Hello

Output of Read(9) function is

Hello The

Output of Readline(9) function is

Hello

Output of Readlines function is

['Hello \n', 'This is Delhi \n', 'This is Paris \n', 'This is London \n']

WRITING TO A FILE:

1. There are two ways to write in a file.

1. write() : Inserts the string str1 in a single line in the text file.

File_object.write(str1)

2. writelines() : For a list of string elements, each string is inserted in the text file.
Used to insert multiple strings at a single time.

File_object.writelines(L) for L = [str1, str2, str3]

Example: 1

```
file = open("Employees.txt", "w")

for i in range(3):
    name = input("Enter the name of the employee: ")
    file.write(name)
    file.write("\n")

file.close()

print("Data is written into the file.")
```

Output:

Data is written into the file.

Example: 2

```
file1 = open("Employees.txt", "w")
lst = []

for i in range(3):
    name = input("Enter the name of the employee: ")
    lst.append(name + '\n')

file1.writelines(lst)
file1.close()

print("Data is written into the file.")
```

Output:

Data is written into the file.

FORMAT OPERATOR IN PYTHON

The format operator, % allows us to construct strings, replacing parts of the strings with the data stored in variables. When applied to integers, % is the modulus operator. But when the first operand is a string, % is the format operator.

The format operator % is used for string formatting. It allows you to embed values within a string, making it more readable and organized.

Basic Syntax:

"format_string" % values

Examples:

1. Formatting with Numbers:

```
python
num = 42
formatted_string = "The number is %d" % num
print(formatted_string) # Output: The number is 42
```

2. Formatting with Multiple Values:

```
name = "Alice"
age = 30
formatted_string = "Name: %s, Age: %d" % (name, age)
print(formatted_string) # Output: Name: Alice, Age: 30
```

3. Floating-Point Numbers:

```
pi = 3.14159
formatted_string = "Pi to two decimal places: %.2f" % pi
print(formatted_string) # Output: Pi to two decimal places: 3.14
```

Common Format Specifiers:

- %d or %i: Integer
- %f: Floating-point number
- %s: String
- %x: Hexadecimal (lowercase)
- %X: Hexadecimal (uppercase)
- %o: Octal

Advantages:

- Readability: Embeds values directly within the string.
- Precision: Controls the number of decimal places for floating-point numbers.

There are five different ways to perform string formatting in Python

1. Formatting with % Operator.
2. Formatting with format() string method.
3. Formatting with string literals, called f-strings.
4. Formatting with String Template Class
5. Formatting with center() string method.

1.Format String using % Operator

In the expression “The scrawny stray dog %s gobbled down” % ‘hurriedly’, the %s placeholder within the string is replaced by the value ‘hurriedly’.

Example:1

```
print('The value of pi is: %5.4f' %(3.141592))
```

Output:

The value of pi is: 3.1416

2.Format String using format () Method

A pair of curly braces { } is replaced into a string and called the **str.format()**

Example:

```
print('We all are {}'.format('equal'))
```

Output:

We all are equal.

3.String Formatting with F-Strings

f-string **f“My name is {name}.”** is used to interpolate the value of the name variable into the string. Example:

```
name = 'Ele'
```

```
print(f"My name is {name}.")
```

Output

My name is Ele.

4.String Template Class

Template Class allows us to create simplified syntax for output specification. The format uses placeholder names formed by \$ with valid Python [identifiers](#) (alphanumeric characters and underscores).

The following code imports the Template class from the string module. The Template class allows us to create a template string with placeholders that can be substituted with actual values. Here we are substituting the values n1 and n2 in place of n3 and n4 in the string n.

Example:

```
from string import Template n1 = 'Hello'
n2 = 'Geeks'
n = Template('Hi $n3 This is $n4.') print(n.substitute(n3=n1, n4=n2))
```

Output:

Hi Hello This is Geeks.

5. Format String using center() Method

The center() method is a built-in method in Python's str class that returns a new string that is centered within a string of a specified width.

Example:

```
string = "GeeksForGeeks!" width = 30  
centered_string = string.center(width) print(centered_string)
```

Output:

GeeksForGeeks!

COMMAND LINE ARGUMENTS IN PYTHON

Command line arguments are parameters passed to a program at the time of execution from the command line interface. In Python, you can access these arguments using the sys module.

Accessing Command Line Arguments:

1. Import the sys Module:

```
import sys
```

2. Access the Arguments: The arguments are stored in sys.argv, which is a list where:

- sys.argv[0] is the name of the script.
- sys.argv[1] to sys.argv[n] are the additional arguments passed.

Example:

Create a Python script named example.py:

```
import sys
```

```
# Print the name of the script
```

```
print("Script name:", sys.argv[0])
```

```
# Print all arguments passed
```

```
for i, arg in enumerate(sys.argv):
```

```
    print(f'Argument {i}: {arg}')
```

Run the script from the command line:

```
bash
```

```
python example.py arg1 arg2 arg3
```

Output:

Script name: example.py

Argument 0: example.py

Argument 1: arg1

Argument 2: arg2

Argument 3: arg3

Using argparse for More Complex Parsing:

For more advanced command line argument parsing, you can use the argparse module.

```
import argparse
```

```
# Create the parser
```

```
parser = argparse.ArgumentParser(description="Example script")
```

```
# Add arguments
```

```
parser.add_argument('arg1', type=str, help='First argument')
```

```
parser.add_argument('arg2', type=int, help='Second argument')
```

```
# Parse the arguments
```

```
args = parser.parse_args()
```

```
# Access the arguments
```

```
print("First argument:", args.arg1)
```

```
print("Second argument:", args.arg2)
```

Run the script from the command line:

```
bash
```

```
python example.py hello 123
```

Output:

First argument: hello

Second argument: 123

Key Points:

- sys.argv is useful for simple argument parsing.
- argparse provides more flexibility and is better for complex argument parsing.
- Command line arguments are typically used for configuring script behavior, passing input data, or enabling features.

ERRORS:

Errors are the problems in a program due to which the program will stop the execution. On the other hand, exceptions are raised when some internal events occur which changes the normal flow of the program. Two types of Error occur in python.

- **Syntax errors**
- **Logical errors (Exceptions)**

Syntax errors:

When the proper syntax of the language is not followed then a syntax error is thrown. It returns a syntax error message because after the if statement a colon: is missing. We can fix this by writing the correct syntax.

Example

```
# initialize the amount variable
amount = 10000
# check that You are eligible to
# purchase Dsa Self Paced or not
if(amount>2999)
    print("You are eligible to purchase Dsa Self Paced")
```

Output:

```
File "/home/ac35380186f4ca7978956ff46697139b.py", line 4
    if(amount>2999)
        ^
SyntaxError: invalid syntax
```

logical errors(Exception)

When in the runtime an error that occurs after passing the syntax test is called exception or logical type. For example, when we divide any number by zero then the Zero Division Error

exception is raised, or when we import a module that does not exist then Import Error is raised.

Example 1:

```
Python3

# initialize the amount variable
marks = 10000

# perform division with 0 |
a = marks / 0
print(a)
```

Output:

```
Traceback (most recent call last):
  File "/home/f3ad05420ab851d4bd106ffb04229907.py", line 4, in <module>
    a=marks/0
ZeroDivisionError: division by zero
```

In the above example the ZeroDivisionError as we are trying to divide a number by 0.

Some of the common built-in exceptions are other than above mention exceptions are:

Exception	Description
IndexError	When the wrong index of a list is retrieved.
AssertionError	It occurs when the assert statement fails
AttributeError	It occurs when an attribute assignment is failed.
ImportError	It occurs when an imported module is not found.
KeyError	It occurs when the key of the dictionary is not found.
NameError	It occurs when the variable is not defined.
MemoryError	It occurs when a program runs out of memory.
TypeError	It occurs when a function and operation are applied in an incorrect type.

HANDLING EXCEPTIONS IN PYTHON

- Python provides two very important features to handle any unexpected error in your Python programs and to add debugging capabilities in them.
 - Assertions**
 - Exception Handling**

S.No.	Exception Name & Description
1	Exception Base class for all exceptions
2	StopIteration Raised when the next() method of an iterator does not point to any object.
3	SystemExit Raised by the sys.exit() function.
4	StandardError Base class for all built-in exceptions except StopIteration and SystemExit.
5	ArithmeticError Base class for all errors that occur for numeric calculation.
6	OverflowError Raised when a calculation exceeds maximum limit for a numeric type.
7	FloatingPointError Raised when a floating point calculation fails.
8	ZeroDivisionError Raised when division or modulo by zero takes place for all numeric types.
9	AssertionError Raised in case of failure of the Assert statement.
10	AttributeError Raised in case of failure of attribute reference or assignment.
11	EOFError Raised when there is no input from either the raw_input() or input() function and the end of file is reached.
12	ImportError Raised when an import statement fails.

13	KeyboardInterrupt Raised when the user interrupts program execution, usually by pressing Ctrl+c.
14	LookupError Base class for all lookup errors.
15	Index Error Raised when an index is not found in a sequence.
16	KeyError Raised when the specified key is not found in the dictionary.
17	NameError Raised when an identifier is not found in the local or global namespace.
18	UnboundLocalError Raised when trying to access a local variable in a function or method but no value has been assigned to it.
19	EnvironmentError Base class for all exceptions that occur outside the Python environment.
20	IOError Raised when an input/ output operation fails, such as the print statement or the open() function when trying to open a file that does not exist.
21	IOError Raised for operating system-related errors.
22	SyntaxError Raised when there is an error in Python syntax.
23	IndentationError Raised when indentation is not specified properly.
24	SystemError Raised when the interpreter finds an internal problem, but when this error is encountered the Python interpreter does not exit.
25	SystemExit Raised when Python interpreter is quit by using the sys.exit() function. If not handled in the code, causes the interpreter to exit.
26	TypeError Raised when an operation or function is attempted that is invalid for the specified data type.
27	ValueError Raised when the built-in function for a data type has the valid type of arguments, but the arguments have invalid values specified.

28	RuntimeError Raised when a generated error does not fall into any category.
29	NotImplementedError Raised when an abstract method that needs to be implemented in an inherited class is not actually implemented.

Assertions in Python

- Assertions in Python are statements that assert or assume a condition to be true. If the condition turns out to be false, Python raises an `AssertionError` exception. They are used to detect programming errors that should never occur if the code is correct.
- In Python, assertions use the `assert` keyword followed by an expression. If the expression evaluates to `False`, an `AssertionError` is raised. Following is the syntax of assertion –

assert condition, message

Where,

condition – A boolean expression that should be true.

message (optional) – An optional message to be displayed if the assertion fails.

Example:

In the following example, we are using assertions to ensure that the variable "num" falls within the valid range of "0" to "100". If the assertion fails, Python raises an "AssertionError", preventing

further execution of the subsequent print statement –

```
print('Enter marks out of 100:')
num = 75
assert num >= 0 and num <= 100
print('Marks obtained:', num)
num = 125
assert num >= 0 and num <= 100
print('Marks obtained:', num) # This line won't be reached if
assertion fails
```

output:

Enter marks out of 100:

Marks obtained: 75

Traceback (most recent call last):

```
File "/home/cg/root/66723bd115007/main.py",
  line 7, in <module> assert num >= 0 and
num <= 100
```

AssertionError

Handling an exception:

- Exceptions are errors that occur during the execution of a program. Python provides a way to handle these exceptions gracefully, allowing the program to continue running or to terminate properly.

Common Exception Types:

- **ValueError:** Raised when a function receives an argument of the correct type but an inappropriate value.
- **TypeError:** Raised when an operation or function is applied to an object of inappropriate type.
- **IndexError:** Raised when attempting to access an index that is not within the bounds of a list or other sequence.
- **KeyError:** Raised when attempting to access a dictionary with a key that does not exist.

Basic Exception Handling with try and except:

Syntax:

try:

```
# Code that might raise an exception
```

```
risky_code()
```

except ExceptionType as e:

```
# Code to handle the exception
```

```
handle_exception(e)
```

The `try` block contains the code that may raise an exception, and the `except` block handles the error.

Example 1: Handling Division by Zero

```
try:
```

```
    result = 10 / 0 # This will raise a ZeroDivisionError
```

```
except ZeroDivisionError:
```

```
    print("Error: Cannot divide by zero!")
```

Output:

```
Error: Cannot divide by zero!
```

Example:2

try:

```
# Attempt to convert a string to an integer
```

```
number = int("not_a_number")
```

except ValueError as e:

```
print(f"ValueError occurred: {e}")
```

Handling Multiple Exceptions:

Example 1 Handling Multiple Specific Exceptions

try:

```
num = int(input("Enter a number: "))
```

```
result = 10 / num # May raise ZeroDivisionError
```

except ZeroDivisionError:

```
print("Error: Division by zero is not allowed.")
```

except ValueError:

```
print("Error: Invalid input! Please enter a number.")
```

If the user enters 0, it prints:

```
Error: Division by zero is not allowed.
```

If the user enters "abc", it prints:

```
Error: Invalid input! Please enter a number.
```

Example:2

try:

```
number = int("not_a_number")
```

except ValueError as e:

```
print(f"ValueError occurred: {e}")
```

except TypeError as e:

```
print(f"TypeError occurred: {e}")
```

The finally Block:

The finally block is used to define cleanup actions that must be executed under all circumstances.

Example:1

```
try:
    file = open("example.txt", "r")
    data = file.read()
except FileNotFoundError as e:
    print(f"FileNotFoundError occurred: {e}")
finally:
    file.close()
    print("File closed.")
```

The else Block:

The else block can be used to define code that should be executed if no exceptions are raised.

Example:2

```
try:
    number = int("42")
except ValueError as e:
    print(f"ValueError occurred: {e}")
else:
    print("Conversion successful.")
```

Using else and finally

- else executes if no exception occurs.
- finally executes regardless of whether an exception occurred or not.

Example 3: Using else and finally

```
try:
    num = int(input("Enter a number: "))
    result = 10 / num
except ZeroDivisionError:
    print("Cannot divide by zero!")
else:
    print("Result:", result)
finally:
    print("Execution complete.")
```

If the user enters 5:

Result: 2.0 Execution complete.

If the user enters 0:

Cannot divide by zero! Execution complete.

Raising Exceptions:

You can raise exceptions manually using the raise keyword.

Example:1

```
def check_positive(number):  
    if number < 0:  
        raise ValueError("Number must be positive")  
    return True  
  
try:  
    check_positive(-5)  
except ValueError as e:  
    print(f'Exception raised: {e}')
```

Example 2: Raising an Exception

```
try:  
    age = int(input("Enter your age: "))  
    if age < 0:  
        raise ValueError("Age cannot be negative!")  
except ValueError as e:  
    print("Error:", e)
```

If the user enters -5:

Error: Age cannot be negative!

Key Points:

- Exceptions allow for robust error handling.
- Use try, except, else, and finally blocks to manage errors.
- Handle specific exceptions to provide meaningful error messages.
- Use the raise keyword to raise exceptions manually.

MODULES IN PYTHON

1. What are modules?(or) Write a note on modular design. **(2 MARKS) (JAN 2018)**
2. Write a Python script to display the current date and time. **(2 MARKS) (JAN 2018)**
3. How do you use command line arguments to give input to the program? (or)
What is command line argument? **(2 MARKS)(MAY 2019)**

- A module is a file containing Python definitions and statements. It allows you to logically organize your Python code. By grouping related code into a module, you can manage and reuse your code more efficiently.
- A module in Python is simply a file containing Python code—functions, classes, or variables—that can be reused in other programs. Modules help in organizing and structuring code efficiently.

Types of Modules

1. Built-in Modules – Provided by Python (e.g., math, random, os)
2. User-defined Modules – Created by users
3. Third-party Modules – Installed via pip (e.g., numpy, pandas)

1. Built-in Modules Example

- Python comes with many built-in modules, such as math, random, and datetime.

Example: Using the math Module

```
import math  
  
print(math.sqrt(25)) # Square root  
print(math.factorial(5)) # Factorial  
print(math.pi) # Value of pi
```

Output:

```
= RESTART: C:/Users/USER/Desktop/w.py  
5.0  
120  
3.141592653589793
```

2. User-defined Module

- You can create your own module by writing Python code in a separate file.

Creating a Module:

Create a file named mymodule.py:


```
# mymodule.py  
def greet(name):  
    return f"Hello, {name}!"  
  
PI = 3.14159
```

Importing a Module:

You can import and use the module in another script:

```
import mymodule  
  
print(mymodule.greet("Alice")) # Output: Hello, Alice!  
print(mymodule.PI) # Output: 3.14159
```

Importing Specific Attributes:

- You can import specific functions or variables from a module:

```
from mymodule import greet, PI  
  
print(greet("Bob")) # Output: Hello, Bob!  
print(PI) # Output: 3.14159
```

Using as to Rename a Module

```
import math as m print(m.sqrt(9)) # Output: 3.0
```

Third-party Modules

- Third-party modules can be installed using pip. Example: Using the numpy module.

```
import numpy as np  
arr = np.array([1, 2, 3, 4])  
print(arr * 2) # Multiply each element by 2  
  
Install numpy using:  
pip install numpy
```

The `__name__ == "__main__"` Concept

- Python modules can be run as standalone scripts or imported into other scripts. The `name_variable` helps distinguish the execution mode.

Example

```
# module_example.py  
def main():  
    print("This is a module!")  
  
if __name__ == "__main__":
```

`main()`

If you import `module_example` elsewhere, `main()` will not execute automatically.

The `dir()` Function

- The `dir()` function lists all functions, classes, and variables in a module.

```
import math  
  
print(dir(math)) # Lists all attributes of math module
```

PACKAGES IN PYTHON

A package is a collection of modules organized in directories. Packages allow you to structure your Python project into multiple modules that can be reused across various projects.

Creating a Package:

1. **Create a Directory:**
 - Let's create a directory named `mypackage`.
2. **Add an `__init__.py` File:**
 - Create an empty file named `__init__.py` inside the directory. This file indicates that the directory should be treated as a package.
3. **Add Modules:**
 - Create two modules inside the `mypackage` directory:
 - `mymodule1.py`
 - `mymodule2.py`

Example Structure:

```
mypackage/  
  __init__.py  
  mymodule1.py  
  mymodule2.py
```

Using a Package:

You can import modules from a package:

```
from mypackage import mymodule1, mymodule2
```

```
# Use functions or variables from the modules
```

```
print(mymodule1.some_function())
```

```
print(mymodule2.another_function())
```

Key Points:

- **Modules:** Single files containing Python definitions and statements. They help in organizing code logically.
- **Packages:** Collections of modules organized in directories. They allow you to structure your project and manage large codebases.
- Use the import statement to bring in modules and packages to your script.
- `__init__.py` files are necessary for creating packages and should be present in each directory you want to treat as a package.